# Smart Library Operations Optimizer

What each console menu option does, in plain terms — and how it maps to the DCIT 308 project brief

---

This app is a Ghana-context library service system: a real SQLite database holds shelves, aisles, book requests, and staff, and every custom data structure and algorithm the brief requires is used to answer a real operational question about that data. Run the jar with `java -jar target/smart-service-optimizer.jar` and pick a number from the menu below. Option 1 must be run first — everything else reads from the database it creates.

## 1. Initialize database and load seed data

### What it does

Creates the database tables (if they don't already exist) and copies the sample data from the CSV files in `data/` into it — 52 shelves, 107 aisles, 320 book requests, and 32 staff/resources. Safe to run more than once: if the data is already loaded, it just says so and does nothing further instead of erroring.

### Built from

DatabaseConnection, DatabaseInitializer, CsvLoader, and all five DAOs (ShelfDAO, PathDAO, BookRequestDAO, StaffDAO, AlgorithmRunDAO).

### Brief requirement it satisfies

Module M2 (database and data loader) and Section 4 — the brief requires the database to be part of the running system, not just storage, and to hold at least 50 locations, 100 roads, 300 requests, and 30 resources.

## 2. Dispatch pending book requests (FIFO/urgency/shift/interrupt)

### What it does

Answers “which request should be handled next?” four different ways using four different structures: oldest-first (a plain queue), most-urgent-first (a priority heap), fixed-size batches per shift (a circular queue), and urgent walk-ins jumping the line (a double-ended queue). Each produces its own ordering of the same 159 pending requests, so you can see how the choice of structure changes the outcome.

### Built from

LinkedQueue, BinaryHeapPriorityQueue, CircularQueue, and CustomDeque, all built from scratch.

### Brief requirement it satisfies

Module M5 (service scheduling engine) and Section 3.i — “which service request should be handled next under FIFO, urgency, and priority-based rules?”

## 3. Search shelf/request indexes (category/section/status)

### What it does

Builds three separate lookup indexes — one over shelf categories, one over shelf sections, one over request statuses — and prints each as a sorted, duplicate-free list. It then shows every shelf in one sample category as a concrete example of a fast lookup.

### Built from

BinarySearchTree (categories), RedBlackTree (sections), and BTree (statuses), each backed by a hash-map grouping for the actual record lookup.

### Brief requirement it satisfies

Module M6 (indexing engine) — “use BST, balanced tree, and B-tree ideas to support search indexes over requests, locations, or resources.”

## 4. Find routes between shelves (fastest path, reachability, MST)

### What it does

Treats the library's shelves and aisles as a map. It reports how many shelves are reachable from the front desk, calculates the shortest walking distance to every other shelf, and works out the cheapest possible layout of aisles that still connects every shelf to every other shelf (useful for planning renovations or emergency routes).

### Built from

AdjacencyListGraph plus BFS, DFS, Dijkstra, Prim, and Kruskal, all built from scratch.

### Brief requirement it satisfies

Module M7 (graph route engine) and Section 3.ii/iii — “what is the fastest route between two locations?” and “which locations are reachable from the current dispatch point?”

## 5. Optimise staff assignment and request selection (greedy/DP)

### What it does

Two different optimisation strategies on the same pending requests: a greedy pass that matches each request to the best available staff member one at a time (fast, but not always the best possible overall outcome — see the note below), and a dynamic-programming pass that picks the highest-value combination of requests that fits inside a fixed staff-time budget.

### Built from

GreedyAssignment and KnapsackDP, both built from scratch.

### Brief requirement it satisfies

Module M8 (optimisation engine) and Section 3.iv — “which subset of requests or resources can be selected under a budget, capacity, or time constraint?” The brief also requires a worked example of greedy failing to find the optimal answer; that proof lives in GreedyAssignmentTest, not in this menu option itself.

## 6. Dispatch a request and undo it (audit trail demo)

### What it does

Picks one pending request, marks it “in progress,” then immediately reverses that change back to its original status — demonstrating that every action can be undone. Both the dispatch and the undo are written to a permanent audit log so there's a record of exactly what happened and when.

### Built from

ArrayStack (holds the undo history) plus the new AuditEvent model and AuditEventDAO.

### Brief requirement it satisfies

Section 6 system diagram, which explicitly calls for a “stack undo and audit trail” feature backed by the audit_events table.

## 7. Run efficiency lab (sort/search/hash experiments)

### What it does

Times every sorting algorithm, both search algorithms, the hash table, the heap, all three trees, and Dijkstra's algorithm at five different input sizes (100 up to 20,000), three repeated runs each. Results are written as raw timing data (CSV) and as line graphs (PNG) under `results/`, so you can see empirically how each algorithm's runtime grows as the amount of data grows.

### Built from

All six sort/search/hash/heap/tree/graph experiment classes, plus CsvExporter and ChartExporter.

### Brief requirement it satisfies

Section 9 (efficiency analysis) and Section 3.v — “how do alternative data structures and algorithms perform as the dataset grows?” The brief's minimum (six structures compared, five input sizes, three runs each) and its named required graphs (sorting time, search time, hash-collision impact, Dijkstra graph-size effect, heap scaling) are all produced here.

---

Option **0** exits the program. Option 1 must be run once before options 2, 3, 4, 5, or 6 will show real results; option 7 works independently since it generates its own test data.